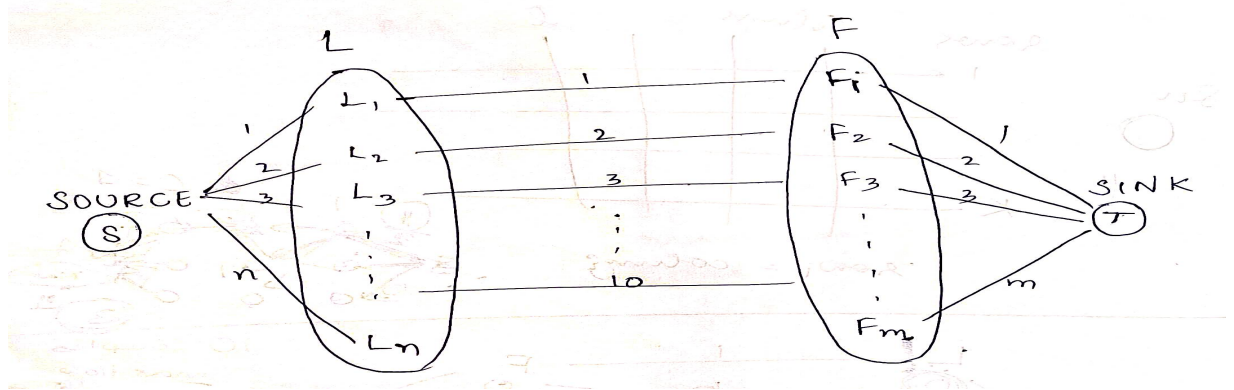


1 ans.

1. Here, there are two vertices, source and sink. The source is connected to each leader(L) and each follower(F) is connected to sink. The number of edges from source is n (amount of leader) and from sink is m (number of followers). The capacity of source is n and capacity of sink is m . Each leader is connected to maximum of 1 follower or each follower can be connected to maximum of 1 leader. So, each dancer and leader will have the maximum capacity of 1. The connections between the leader and follower are the possible number of couples that can pair up. It represents the at most couples that can group up.
2. The edge is the connection between the source to leader, leader to follower and follower to sink. The necessary connection is the source to capacity of source (number of leaders) and sink to capacity of sink (number of followers). It means that we have to calculate at least the number of nodes that we have connect. That will be $m + n$.
3. Since only one leader can pair up with only one follower. And, at most only 10 couples can dance. Even if we assume there are more than 10 leaders and 10 followers, from them only 10 from each can be together, that implies all the other are ignored. So, at any point of time how many flows we have are the maximum available dance pairs. We can assume less if leaders and followers are not the same and if they are below 10, but the maximum flow would be 10.



2 ans.

We have K rows and I columns. Such that the sum of some k rows is equal to sum of some I columns.

We will use the dynamic programming, in which we can store the sum of each rows and columns of the given values. Here we iterate the rows and columns. It repeats until the sum values of each row or column matches with at least one of other. Otherwise, it turns says as not matched.

As in Dynamic program we save the sum of each row and column values. We check if the $\text{row}(i)$ value matches with the sum of r_i . Similarly, we do the same with the column. Check the sum of $\text{column}(j)$ values with the c_j . This we can do by using else statement. If the value of the sum of i^{th} row or j^{th} column matches with its respective r_i or c_j , then it returns true otherwise it returns false.

3 ans

We have in the graph with the vertex's, edge's, source, sink, capacity and degree. We have the capacity for each is 1.

1. Here, the vertices are 11. S and T are two, so remaining nodes are 9. $D=3$ (constant).

(a) The maximum flow from the above graph:

The path as goes: $s-4-5-8-9-8-7-5-6-9-t = 1$ (maximum flow, low value)

As its capacity is 1, it can send only 1 unit.

(b) we consider different path and add up the value.

The three paths : $s-1-2-3-t = 1$

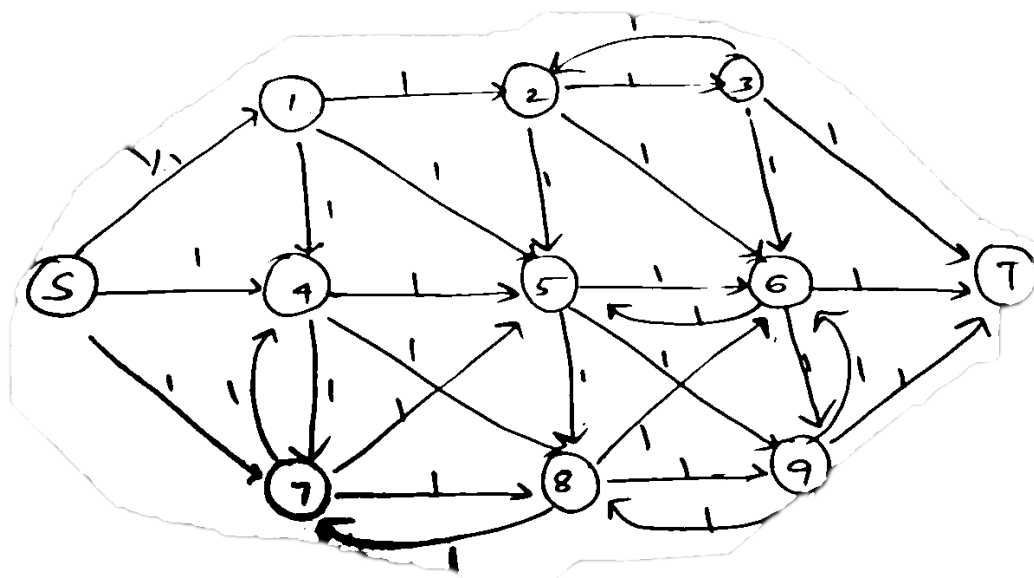
$s-4-8-9-t = 1$

$s-7-5-6-t = 1$. They add up to 3.

So, the maximum value will be 3.

2. number of edges = $n \times d$

$FF = \text{bigO}((n \times (n \times d))^2)$



4 ans.

1. So, we have a source(s) and sink(T). The S and T can be found by performing the max flow. The vertex (v) is said to be on S-node or on T-node. This can be found by calculating the max flow of the graph. The sets of S and T can be figured out in this method. From S to each node, we can path the max flow. When the max flow is equal to the capacity of the

vertex node connected with the source, then it is called as S-node. The max flow from the vertex stops at the sink(T). If, the max flow is equal to the capacity of the vertex that is connected to the sink, then it is called as T-node. If it does not match with any of the vertex's capacity connected to any of the source or sink, then it is neither.

2. From the first step, with the max flow algorithm the nodes of S and T are found. That is the Vertex's that are S-node and T-node are found. So, the vertex is said to be S-node when the capacity of the source to vertex is equal to the max flow. The max flow can also be considered with the complete path from the vertex to the sink. And, if this is equal with the vertex to sink capacity, then it is called as T-node. Times when is matched with both the S and T vertex capacity, then there would be different minimum st-cuts. Except in a graph there will be only one possible minimum st-cut.